HOSPITAL UNIVERSITARIO

GUÍA PARA EL EQUIPO DE SISTEMAS

HU-KIT

Qué es, cómo se ve un sistema del hospital, cómo se arranca uno nuevo y cómo se trabaja con él todos los días.

Versión 1.8.1 · octubre de 2026

HOSPITAL UNIVERSITARIO

Tres piezas, cada una con su trabajo

Todos los sistemas del Hospital Universitario se tienen que ver igual, comportarse igual y estar programados de la misma forma, sin copiarse código entre sí. Para eso hay tres piezas:

SE INSTALA

hu-kit

La caja de herramientas. Los colores, las fuentes y las piezas de pantalla: botones, formularios, tablas, el menú lateral, los avisos.

Cada sistema lo instala con npm. Cuando hu-kit cambia, cada sistema decide cuándo sube de versión.

SE COPIA UNA VEZ

hu-plantilla

El molde. Un sistema vacío pero completo: login, menú, roles, tests y documentación, con hu-kit ya instalado.

Se copia al arrancar un sistema nuevo y después cada uno sigue por su lado. *Todavía no existe: es el próximo paso.*

LO PROPIO

Cada sistema

Lo que lo hace único. Sus pantallas, su menú, sus datos, sus reglas: quirofanohu, Bienes HU, el próximo.

Sólo escribe lo suyo: el resto lo trae hu-kit.

La diferencia entre instalar y copiar

Lo que se **instala** sigue siendo igual en todos los sistemas: si mañana cambia un botón en hu-kit, todos lo reciben al subir de versión. Lo que se **copia** es un punto de partida que cada sistema después adapta, como sus roles o su menú. Una pieza va a hu-kit cuando *tiene que* seguir igual; va a la plantilla cuando alcanza con que *arranque* igual.

El catálogo: la vidriera de hu-kit

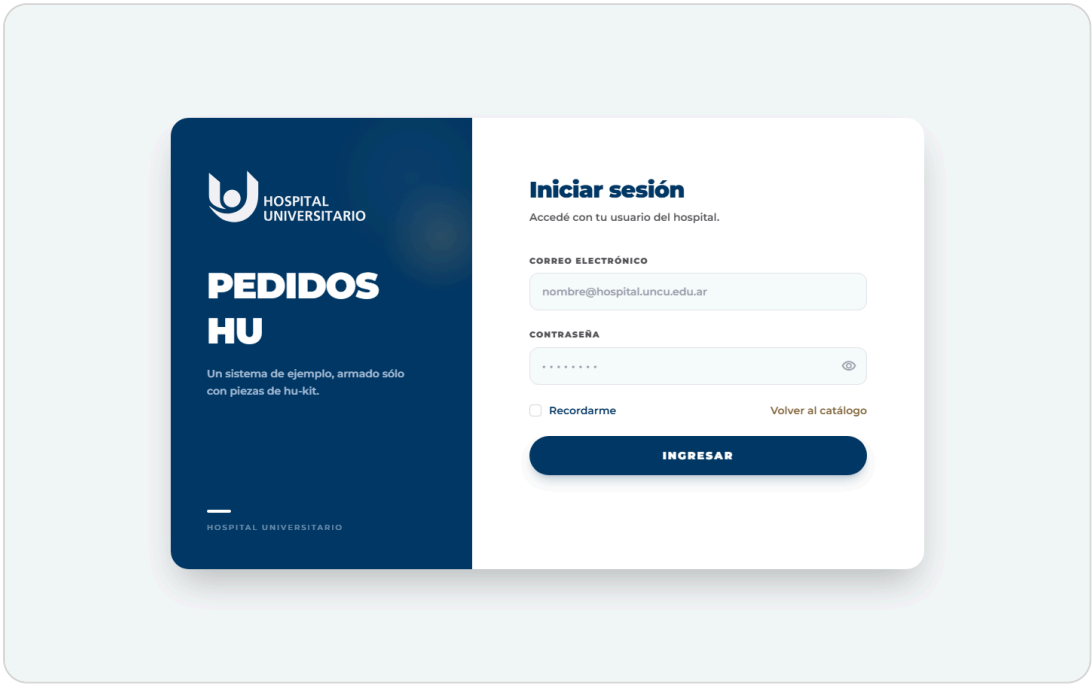
Es una página que vive dentro de hu-kit y muestra armadas todas las piezas, una al lado de la otra, en claro y en oscuro. No tiene base de datos ni login: los datos son inventados. Sirve para mirar qué hay antes de armar una pantalla, para probar un cambio en hu-kit sin instalarlo en un sistema, y de ahí salen las capturas de esta guía.

```
cd C:\laragon\www\hu-kit
npm install          (la primera vez)
npm run catalogo     (y abrir en el navegador la dirección que muestra)
```

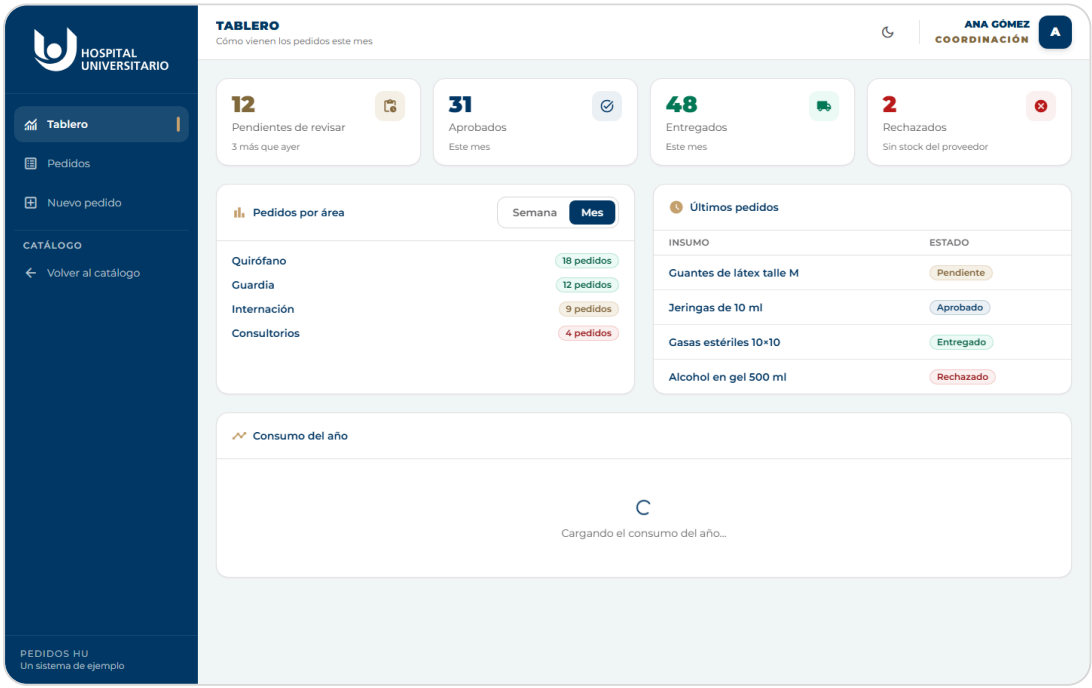
2 · CÓMO SE VE

Un sistema del HU, de punta a punta

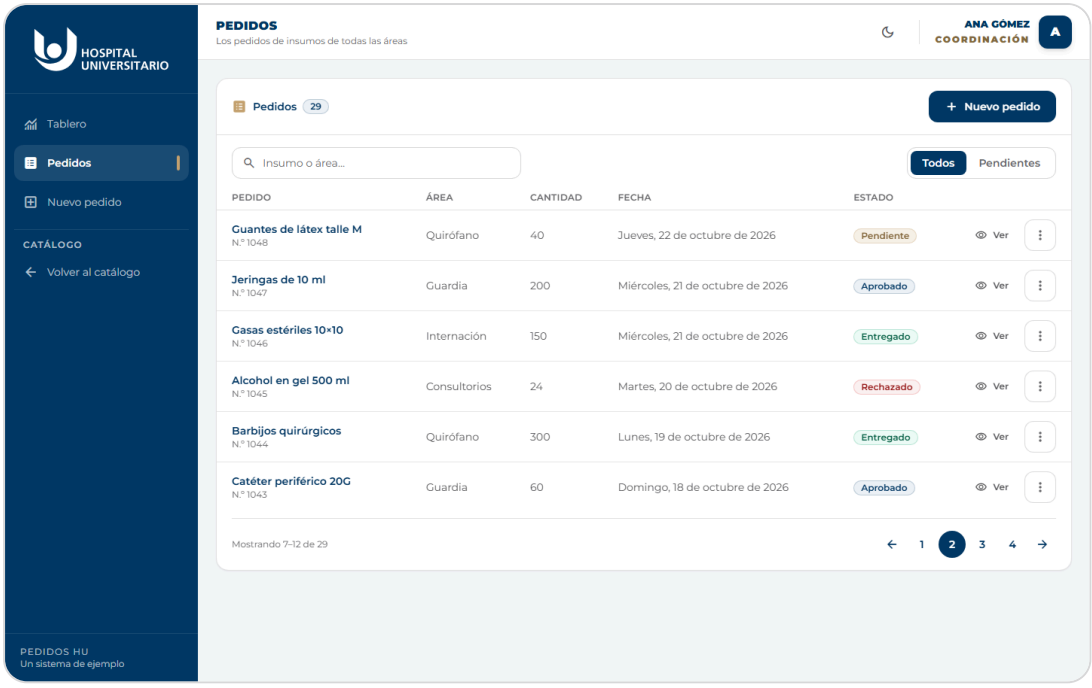
Estas pantallas son de «Pedidos HU», un sistema inventado que vive en el catálogo. Está armado **sólo con piezas de hu-kit**: ningún color, botón ni menú está hecho a mano. Así se va a ver cualquier sistema del hospital.



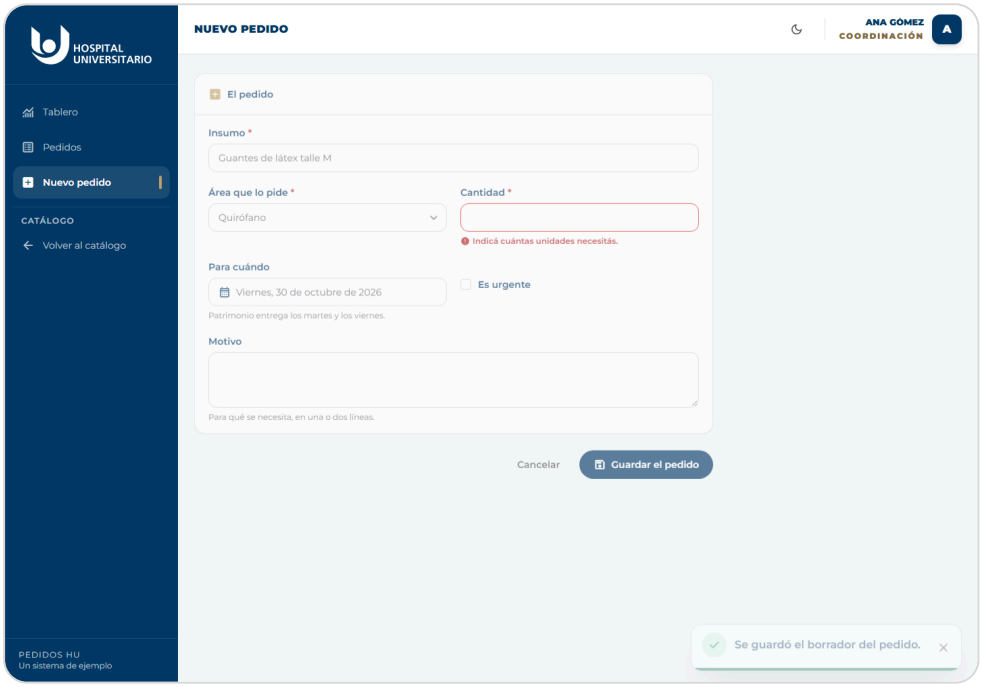
El ingreso. `PanelAutenticacion`, copiado del login del portal de pacientes. Cada sistema le pasa su nombre y su bajada.



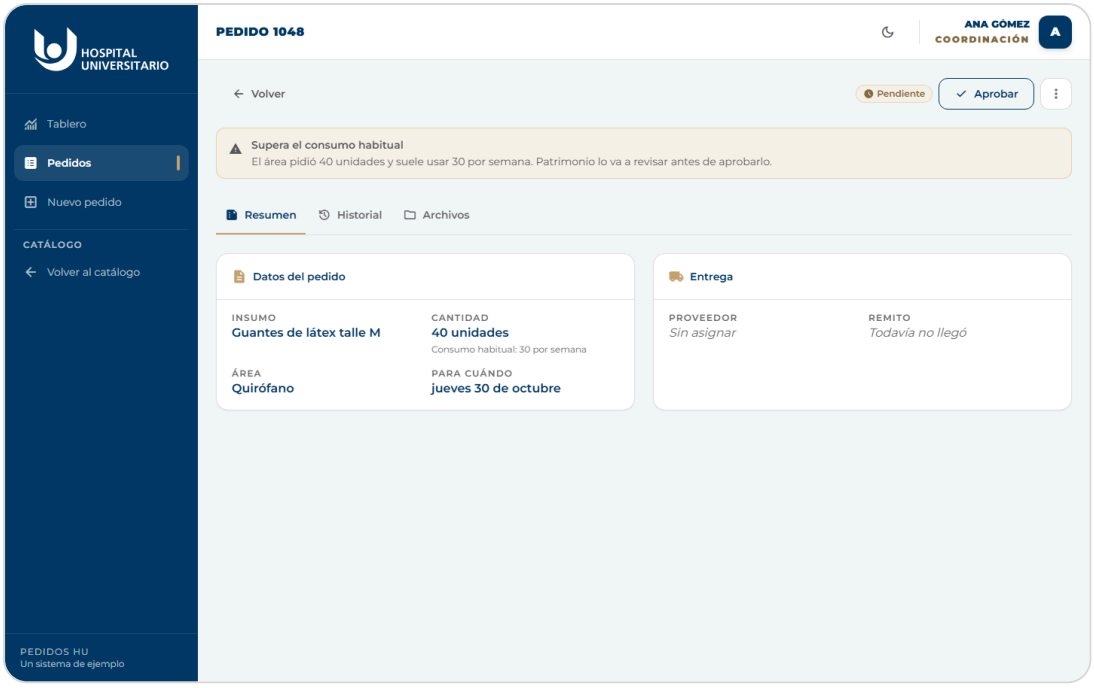
El tablero. El marco (`MarcoApp`: barra lateral, encabezado, menú de la cuenta) con `Metrica`, `Tarjeta`, `Estado`, `SelectorDeVista` y `Cargando`.



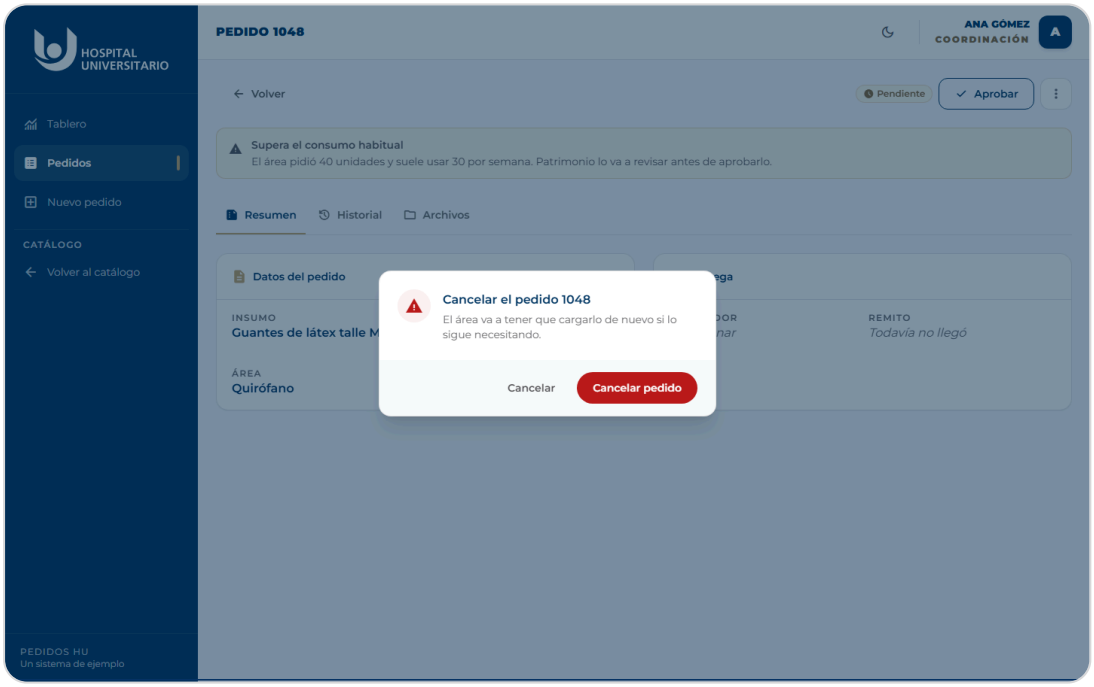
Un listado. `Tarjeta` con contador y acción, `Buscador`, `SelectorDeVista`, la tabla de hu-kit, `Estado`, `MenuDeAcciones` en cada fila y `Paginador`.



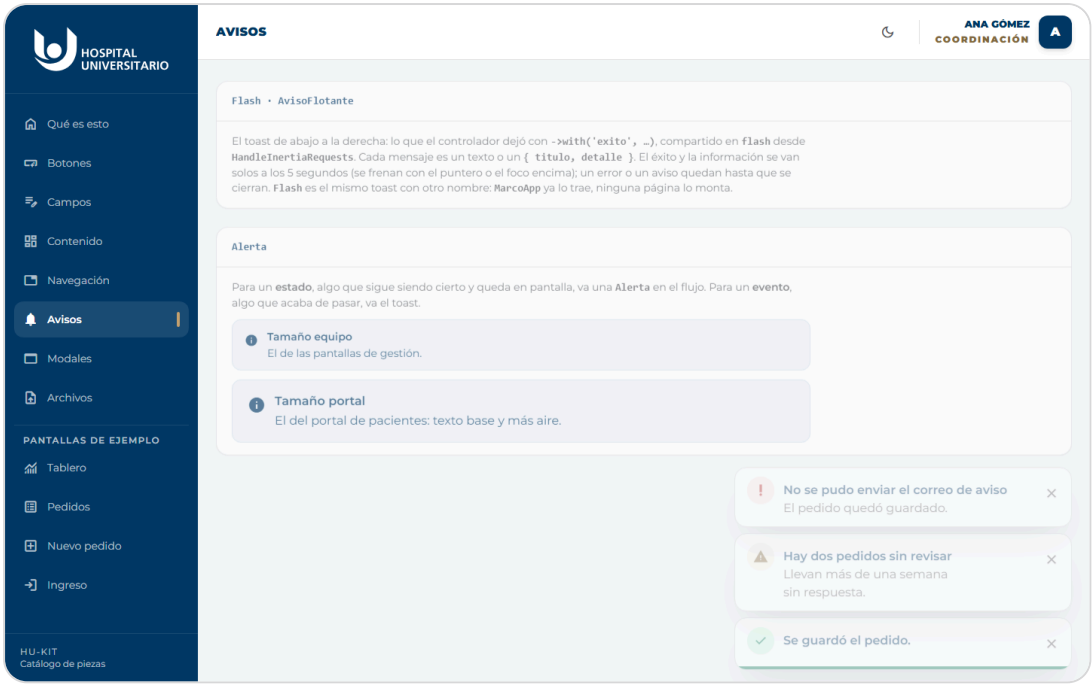
Un formulario. `Input`, `Select`, `SelectorFecha`, `Checkbox` y `Textarea`, con su etiqueta, su ayuda y su error. Abajo a la derecha, el aviso de `Flash` que dejó el servidor.



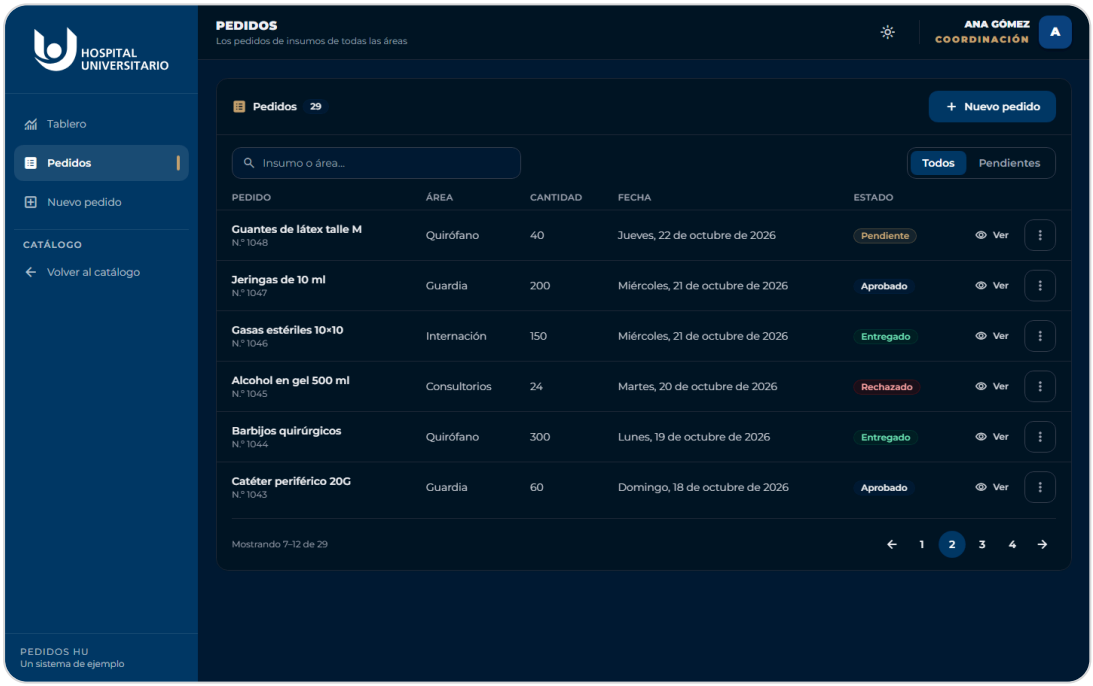
Un detalle. `Solapas` para las partes del pedido, `Alerta`, `Dato` para cada valor de la ficha y `MenuDeAcciones` para lo que no entra como botón.



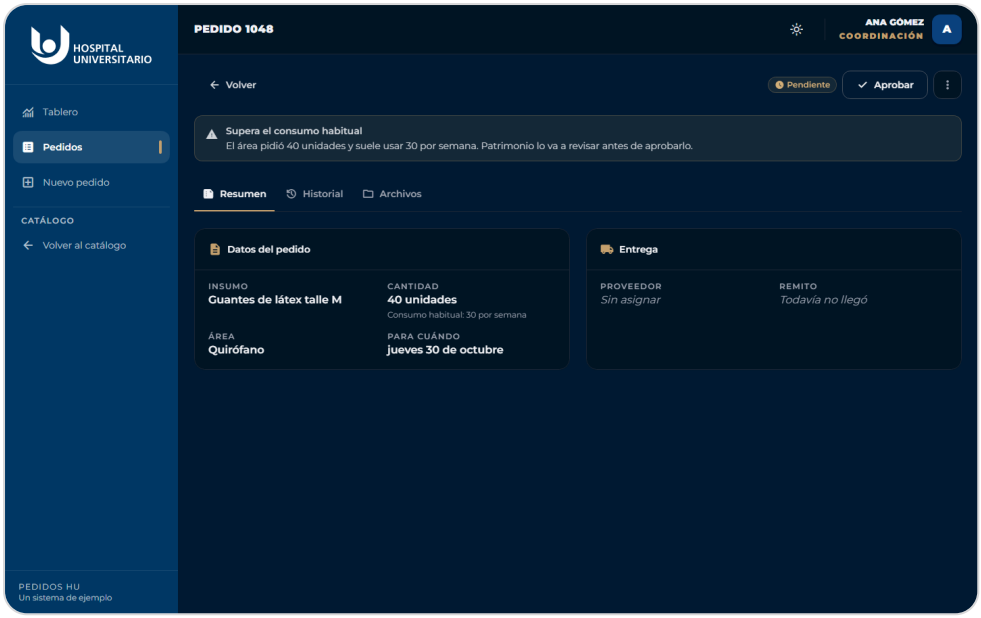
Confirmar antes de borrar. `ModalConfirmar` con `useConfirmacion`: toda acción destructiva pregunta antes. Nunca `window.confirm()`.



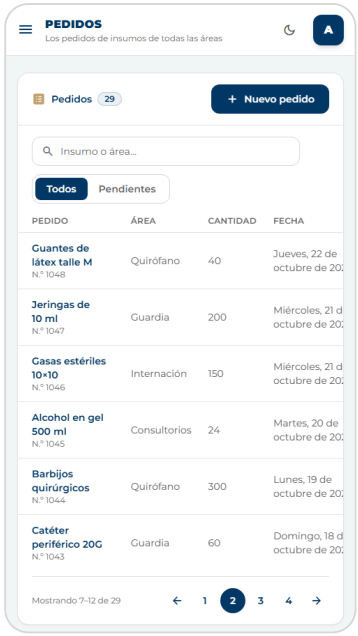
Los avisos. Abajo a la derecha, el toast (`Flash` o `AvisoFlotante`) con los cuatro tipos (éxito, error, aviso, información), con título y detalle. Dentro de la pantalla, `Alerta` en sus dos tamaños.



El modo oscuro. Lo trae cada pieza: el sistema no escribe nada para tenerlo. No se guarda entre sesiones, porque las computadoras del hospital son compartidas.



El detalle, en oscuro.



En un celular. El menú lateral se esconde detrás del botón de arriba a la izquierda, y la tabla se desplaza de costado.

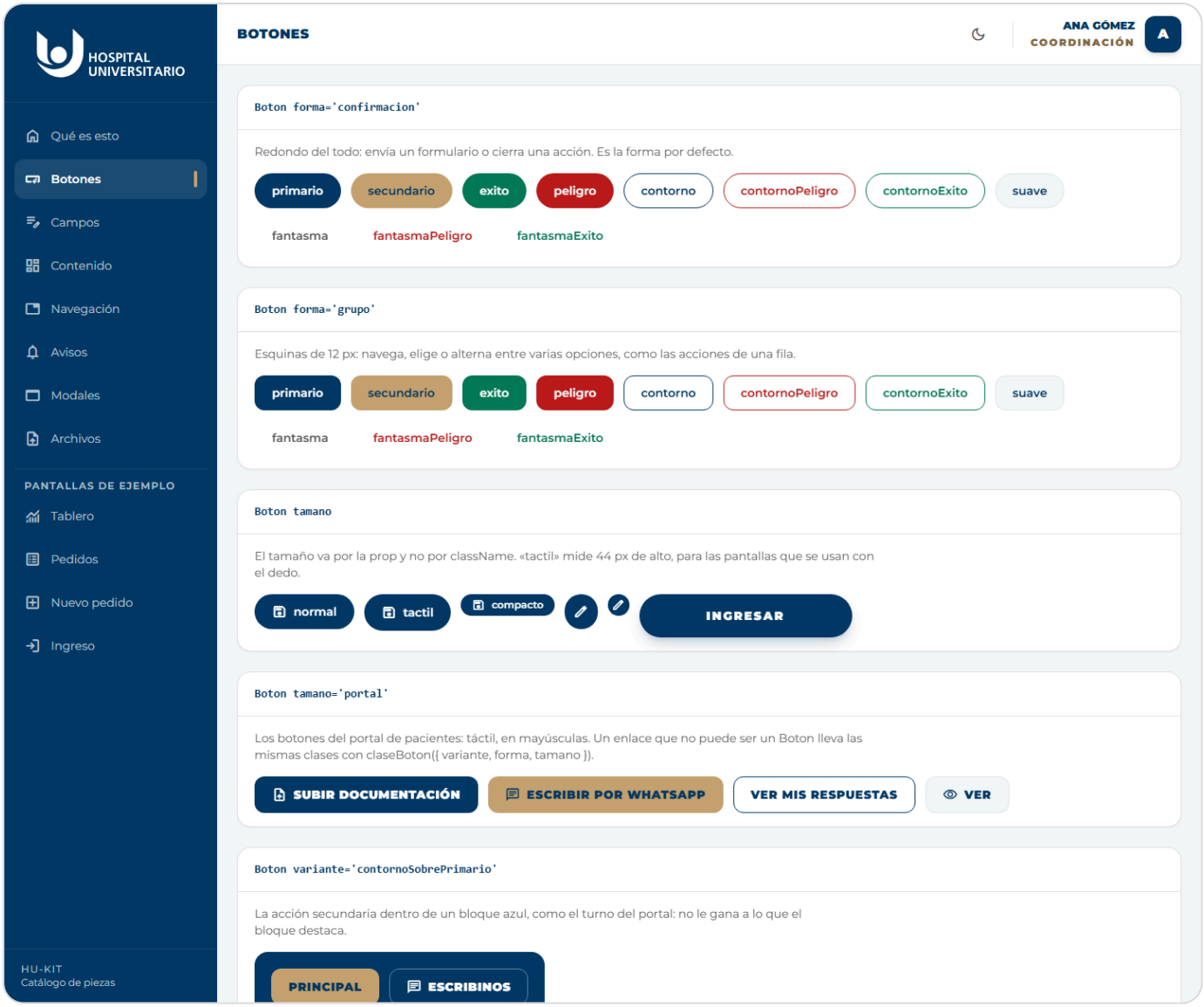
Qué hay en la caja

Todo se importa de **`hu-kit/componentes`**. Si lo que vas a armar está acá, se importa: copiarlo es lo que hace que dos sistemas terminen distintos.

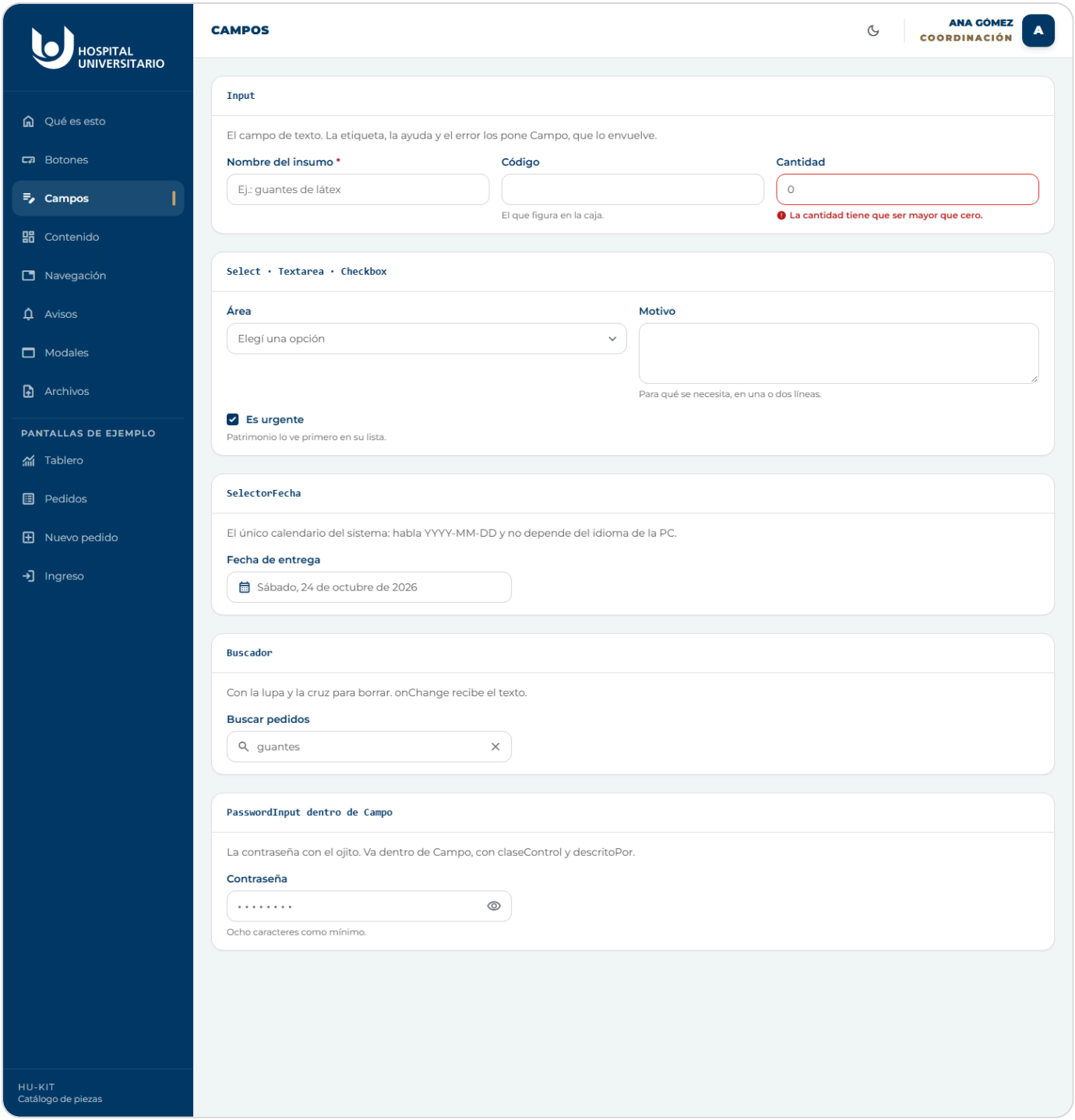
```
import { Boton, Tarjeta, Estado } from 'hu-kit/componentes';
```

PIEZA	QUÉ ES Y CUÁNDO SE USA
MarcoApp	El marco de toda pantalla con sesión: barra lateral, encabezado, menú de la cuenta y avisos. Cada sistema lo envuelve en su `AppLayout`.
PanelAutenticacion	La pantalla de ingreso.
Boton	Todos los botones. Redondo del todo para confirmar o enviar; esquinas de 12 px para navegar o elegir.
Input · Select · Textarea · Checkbox	Los campos de un formulario, con su etiqueta, su ayuda y su error.
SelectorFecha	El único calendario. Nunca el `type="date"` del navegador.
Buscador	El campo de búsqueda con lupa y botón para borrar.
PasswordInput · Campo	La contraseña con el ojito, adentro de `Campo`, que pone la etiqueta y el error a un control propio.
Tarjeta	El contenedor de todo contenido, con título, contador y acciones.
Estado	La píldora de estado: éxito, error, aviso, información o neutro.
Alerta	Un estado que sigue siendo cierto y queda en la pantalla. Tamaño `equipo` o `portal`.
EstadoVacio · Cargando	Lo que se ve cuando no hay nada, y mientras algo llega.
Dato · Rotulo · Metrica	Un valor de una ficha; el rótulo chico en mayúsculas; el número grande de un tablero.
Tabla · Paginador	Las clases de una tabla de datos y la paginación, a partir del paginador de Laravel.
Solapas · SelectorDeVista	Las partes de una pantalla larga; ver lo mismo de otra forma (mes, semana, día).
MenuDeAcciones	Las acciones que no entran como botón, detrás de los tres puntos.
Flash · AvisoFlotante	El toast con los mensajes que deja el servidor después de una acción: éxito e información se van a los 5 s; error y aviso quedan hasta cerrarlos. `Flash` es el mismo con otro nombre.

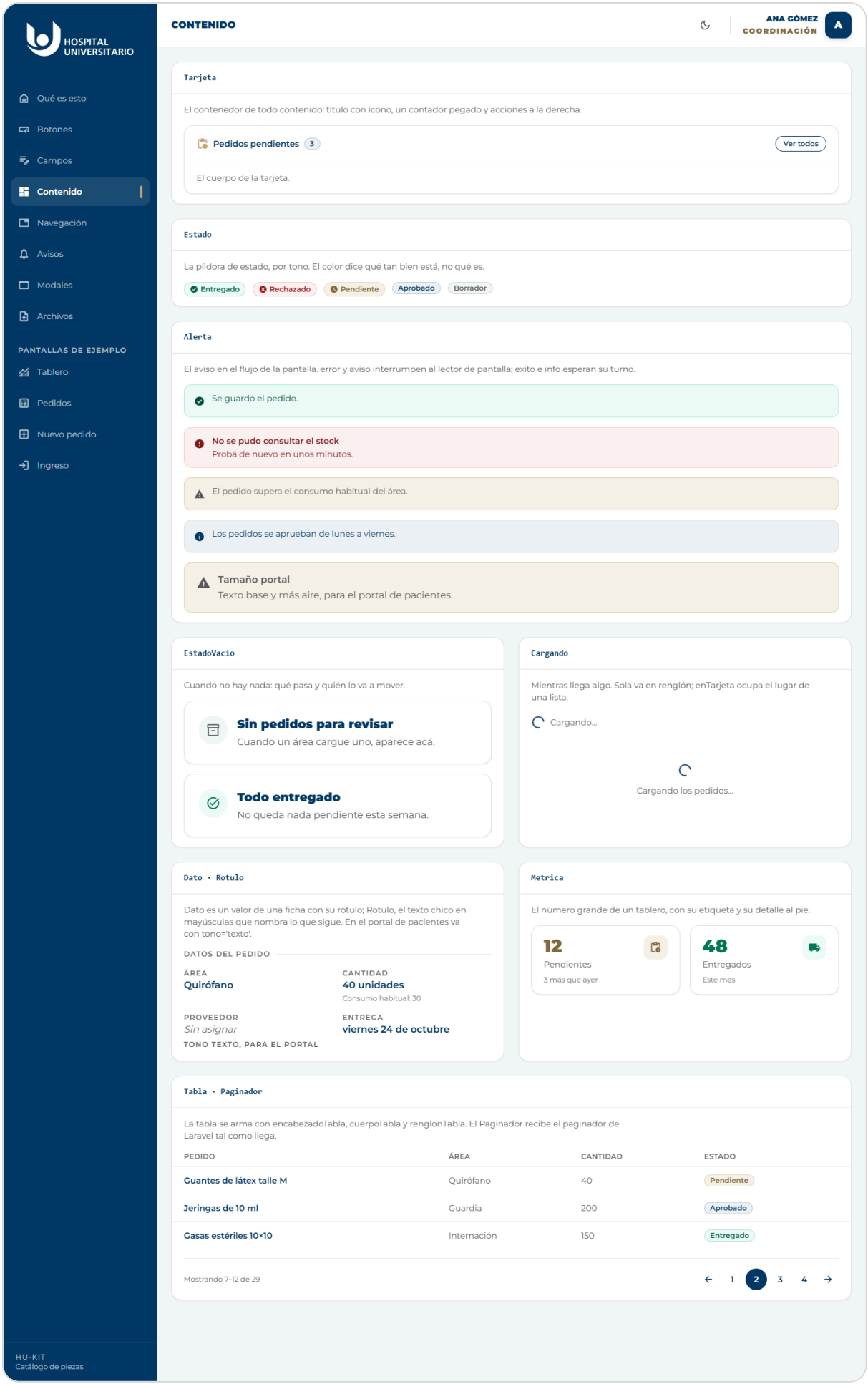
PIEZA	QUÉ ES Y CUÁNDO SE USA
Modal · ModalConfirmar	Un formulario corto en un diálogo; la confirmación de una acción destructiva.
SelectorDeArchivo · BarraDeProgreso	Elegir un archivo (en el celular, también sacar una foto) y ver cuánto lleva la subida.
Icono · ApplicationLogo	Los íconos de Material Symbols y el logo del hospital.
institucional	La dirección, el teléfono, el correo y el WhatsApp de turnos del hospital, en un solo lugar.



Botones, en el catálogo: sus dos formas, sus colores y sus tamaños.



Campos, con su ayuda y su error.



Contenido: tarjetas, estados, alertas, vacío, cargando, datos, métricas y tabla.

Arrancar «Farmacia HU»

Supongamos que hay que hacer un sistema nuevo para Farmacia. Así se arranca.

Con hu-plantilla · cuando exista

1. En GitHub, en `mauroas7/hu-plantilla`, tocar **«Use this template»** y crear el repo `farmaciahu`.
2. Clonarlo e instalarlo:

```
git clone https://github.com/mauroas7/farmaciahu.git C:\laragon\www\farmaciahu
cd C:\laragon\www\farmaciahu
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate --seed
npm install
npm run dev
```

3. Abrir `http://farmaciahu.test` (Laragon lo publica solo): ya aparece el ingreso del HU, con el menú lateral y el modo oscuro andando.
4. Cambiar las cinco cosas propias del sistema:
   - el nombre, en `.env` (`APP_NAME`);
   - el texto del ingreso, en `Components/UI/PanelDeIngreso.jsx`;
   - las opciones del menú, en `data/navegacion.js`;
   - los roles, en `app/Enums/Rol.php`;
   - el `README.md`.
5. `php artisan test` tiene que dar verde: trae los tests que vigilan las convenciones.
6. Para la primera pantalla, abrir el catálogo, elegir las piezas y armar la página con ellas. No hay colores ni estilos que decidir.

Mientras no exista la plantilla

Es el camino largo, el que siguió Bienes HU: un Laravel con Inertia y React (por ejemplo, con Breeze), sacarle lo que no se usa y conectarle hu-kit. Conectar hu-kit son cuatro archivos:

```
npm install hu-kit@github:mauroas7/hu-kit#v1.8.1
```

```
// tailwind.config.js
import huKit from 'hu-kit/tailwind';

export default {
    presets: [huKit],
    content: [
        './resources/views/**/*.blade.php',
        './resources/js/**/*.{js,jsx}',
        './node_modules/hu-kit/componentes/**/*.{js,jsx}',
    ],
};
```

```
/* resources/css/app.css */
@import 'hu-kit/estilos.css';

@tailwind base;
@tailwind components;
@tailwind utilities;
```

```
// resources/js/app.jsx: la red de errores, el modo oscuro y el color de la barra de carga
import { ErrorBoundary, ThemeProvider } from 'hu-kit/componentes';
import { colores } from 'hu-kit/tokens';
…
root.render(
    <ErrorBoundary onError={registrarError}>
        <ThemeProvider usuarioInicial={props.initialPage.props.auth?.usuario?.id ?? null}>
            <App {...props} />
        </ThemeProvider>
    </ErrorBoundary>,
);
…
progress: { color: colores.secondary.DEFAULT },
```

Y el puente `AppLayout` de la página siguiente.

El puente: el AppLayout de cada sistema

`MarcoApp` no sabe nada de un sistema en particular: recibe el menú, el usuario y los enlaces ya armados. Cada sistema tiene un `AppLayout` de unas 25 líneas que se los pasa, y sus páginas usan sólo ese `AppLayout`.

```
// resources/js/Components/UI/AppLayout.jsx
import { usePage } from '@inertiajs/react';
import { MarcoApp, rutaActiva } from 'hu-kit/componentes';
import { seccionesDelRol } from '@/utils/secciones';

const SISTEMA = { nombre: 'Farmacia HU', bajada: 'Pedidos y stock de farmacia' };

export default function AppLayout({ titulo, subtitulo = null, ancho = 'completo', children }) {
    const { usuario } = usePage().props.auth;

    // El menú es de cada sistema: acá se filtra por rol y se le pone a cada opción su dirección.
    const secciones = seccionesDelRol(usuario?.rol).map((seccion) => ({
        ...seccion,
        href: seccion.ruta && route(seccion.ruta),
        activa: Boolean(seccion.ruta) && rutaActiva(seccion.activaEn ?? seccion.ruta),
    }));

    return (
        <MarcoApp
            titulo={titulo} subtitulo={subtitulo} ancho={ancho}
            secciones={secciones}
            inicio={route('inicio')}
            sistema={SISTEMA}
            usuario={usuario && { nombre: usuario.nombre, email: usuario.email, rol: usuario.rolEtiqueta }}
            enlaces={[{ etiqueta: 'Mi cuenta', icono: 'manage_accounts', href: route('cuenta') }]}
            salir={route('logout')}
        >
            {children}
        </MarcoApp>
    );
}
```

```
// resources/js/data/navegacion.js: el menú, sin lógica
export const SECCIONES = [
    { etiqueta: 'Tablero', icono: 'monitoring', ruta: 'tablero', roles: ['farmacia'] },
    { etiqueta: 'Pedidos', icono: 'list_alt', ruta: 'pedidos.index', activaEn: 'pedidos.', roles: [] },
    { etiqueta: 'Usuarios', icono: 'manage_accounts', ruta: 'admin.usuarios.index', titulo: 'Administración',
roles: ['admin'] },
];
```

Lo que el backend tiene que compartir

`HandleInertiaRequests::share()` manda `auth.usuario` (con `id`, `nombre`, `email`, `rol` y `rolEtiqueta`) y `flash` con `exito`, `error`, `aviso` e `info`. Con eso andan el marco, el menú de la cuenta, el modo oscuro y los avisos.

Acoplar hu-kit: el caso de Bienes HU

Bienes HU ya usaba hu-kit, pero tenía su propio menú lateral, copiado de quirofanohu, que en dos días ya se había separado del original. Con la versión 1.8.1 ese marco pasa a hu-kit. Así se hizo, paso a paso:

PASO	QUIÉN	QUÉ PASA
1	Quien mantiene hu-kit	Arma las piezas nuevas y las mira en el catálogo, en claro y en oscuro.
2	Quien mantiene hu-kit	Instala en el sistema una versión de prueba, sin publicar, y cambia sus archivos (tabla de abajo).
3	Los dos	Prueban que compile, que pasen los tests y que el sistema se vea igual que antes. Si algo falla, se corrige en hu-kit y se repite el paso 2.
4	Quien mantiene hu-kit	Publica la versión: commit, tag `v1.8.1` y push. Un tag publicado no se mueve nunca.
5	Quien trabaja en el sistema	Apunta el sistema a la versión oficial y hace su commit.

Los comandos

```
REM paso 2, la versión de prueba (desde hu-kit y desde el sistema)
npm pack --pack-destination %TEMP%
npm install %TEMP%\hu-kit-1.8.1.tgz
npx vite --force

REM paso 5, la versión oficial (desde el sistema)
npm install hu-kit@github:mauroas7/hu-kit#v1.8.1
```

Qué cambió adentro de Bienes HU

ARCHIVO	ANTES	DESPUÉS
`Components/UI/AppLayout.jsx`	119 líneas: el marco entero	El puente a `MarcoApp`
`Components/UI/Nav.jsx`	53 líneas	Se borra: lo trae hu-kit
`Components/UI/MenuDeUsuario.jsx`	69 líneas	Se borra
`Components/UI/Flash.jsx`	17 líneas	Se borra: lo trae hu-kit
`Components/UI/Dato.jsx`	16 líneas	Se borra: `Dato` pasó a hu-kit
`utils/rutas.js`	33 líneas	Se borra: `rutaActiva` está en hu-kit
`data/navegacion.js`	El menú	Igual: el menú es de cada sistema
`Las 11 páginas`	—	No se tocan

El resultado: el sistema se ve igual, pero el marco ya no es una copia. Si mañana cambia el menú de la cuenta en hu-kit, todos los sistemas lo reciben al subir de versión.

Trabajar con hu-kit

Hacer una pantalla

Abrir el catálogo, buscar las piezas que hacen falta e importarlas. Si una pantalla del catálogo se parece a la que vas a hacer, copiá su forma: está en `catalogo/ejemplos/`.

Falta una pieza

La pregunta es una sola: **¿la va a usar otro sistema?**

Sí → va a hu-kit

Se agrega en `componentes/`, con su ejemplo en el catálogo, su fila en el README y su entrada en el CHANGELOG. Se publica una versión nueva y el sistema la instala.

No → queda en el sistema

Se hace en `resources/js/Components/` del sistema, armada con piezas de hu-kit. Si después aparece en un segundo sistema, recién ahí sube.

Lo mismo con un color: si falta un tono, se agrega con nombre en hu-kit. Nunca un color escrito a mano (`bg-[#…]`).

Subir la versión de hu-kit en un sistema

```
npm install hu-kit@github:mauroas7/hu-kit#vX.Y.Z
npm run build
php artisan test
```

Antes, leer en el CHANGELOG de hu-kit qué trae la versión y si pide algo «al subir». Cambiar el número a mano en `package.json` no alcanza: el lock se queda con la versión vieja.

Publicar una versión de hu-kit (quien lo mantiene)

1. Probarla en un sistema con la versión de prueba (`npm pack`), como en el paso 2 de la sección anterior.
2. Subir `version` en `package.json` y anotar la versión arriba de todo en `CHANGELOG.md`.
3. Si cambió alguna pieza, `npm run guia` vuelve a sacar las capturas y este PDF.
4. Commit, tag anotado con el mismo número y push: `git tag -a vX.Y.Z -m vX.Y.Z` y `git push origin main vX.Y.Z`.
5. Si cambiaron las skills de Claude, volver a correr `instalar.ps1`.

Un error en una versión publicada se arregla publicando la siguiente: un tag no se mueve ni se borra.

Dónde va cada cosa

SI LO QUE VAS A AGREGAR ES…	VA EN
Un color, una fuente, un ícono, una capa, una animación	**hu-kit, en el preset**
Una pieza de pantalla que se ve igual en todos y no conoce el dominio	**hu-kit, en componentes**
Una regla de cómo se programa (capas, errores, comentarios)	**hu-kit, en las skills; su test, en la plantilla**
El arranque que después cada sistema adapta: login, roles, tests, docs	**hu-plantilla**
El dominio: cirugías, bienes, pedidos; su menú y sus pantallas	**El sistema**
Documentación privada de terceros, o el Manual de Marca	**hu-referencias (privado)**

La marca, en cinco reglas

primary · #003764
Pantone 2955 C

secondary · #C7A36E
Pantone 465 M

brandText · #59595B
El gris del logotipo

1. **Tres colores, y sus derivados con nombre.** Nunca un color escrito a mano. Todo texto llega a un contraste de 4,5:1: el dorado sobre fondo claro es `secondary-dark`.
2. **Montserrat, en tres pesos:** 400, 600 y 900 (`font-normal`, `font-semibold`, `font-black`). Nunca `font-bold` ni `font-medium`.
3. **Dos formas de botón:** redondo del todo para confirmar o enviar; esquinas de 12 px para navegar o elegir.
4. **Íconos Material Symbols Outlined:** rellenos sólo en la sección en la que estás parado.
5. **Español rioplatense, con voseo:** «podés», «tenés», «elegí». Frases cortas, sin signos de exclamación.

Dónde está cada cosa

hu-kit	`github.com/mauroas7/hu-kit`; en la máquina, `C:\laragon\www\hu-kit`
El catálogo	`npm run catalogo`, dentro de hu-kit
Esta guía	`docs/guia-hu-kit.pdf`, dentro de hu-kit; se regenera con `npm run guia`
El Manual de Marca	`manual-de-marca/`, en el repo privado `hu-referencias`
Qué trae cada versión	`CHANGELOG.md`, dentro de hu-kit
Las reglas de backend y de marca	`skills/arquitectura-hu/` y `skills/marca-hu/`, dentro de hu-kit. Se leen como documentación; quien usa Claude Code las instala con `instalar.ps1`